# Should the Fusion Weight Change with the Question?

## Part 2: Can Continual Learning Learn the Weight? (Q2–Q3)

Longtao Jiang
with Jianxiang He and Makhmud Assentay

CV805 Course Project
Mohamed bin Zayed University of Artificial Intelligence

November 7, 2026



# Team and my part

| Member | Main contribution | |
|---|---|---|
| Jianxiang He | Problem and setup; score cache and data pipeline; Q1 analysis; overall study (Q1–Q4) | |
| **Longtao Jiang** | **Continual learning experiments and analysis (Q2–Q3)** | **this talk** |
| Makhmud Assentay | Downstream question-answering evaluation (Q4) | |

# Background: two clues, one weight

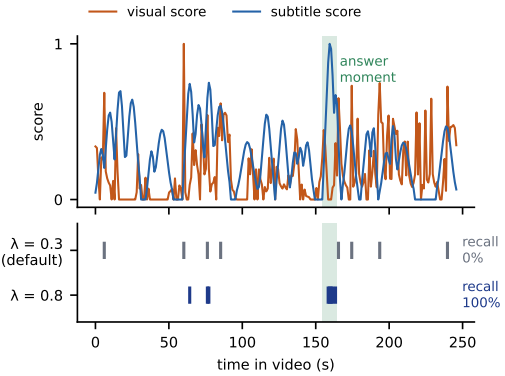
A VLM can only look at **about 8 frames** of a long video, so we must pick them well.

VSI scores every second with two clues:

- **Visual**: do the question's objects appear?
- **Subtitle**: does the subtitle match the question?

$$S(t) = (1 - \lambda)\,\text{visual}(t) + \lambda\,\text{subtitle}(t)$$

and keeps the top 8 seconds. VSI uses $\lambda = 0.3$ **for every question**.




Real example: $\lambda = 0.3$ **misses** the answer moment (green), $\lambda = 0.8$ **hits** it.

# My part: learning the weight

- The best $\lambda$ differs by question type: text questions want **0.8–0.9**, visual questions $\leq$**0.3**.
- Idea: a **small predictor** outputs $\lambda$ for each question.
- New question types keep arriving, so it learns them **one after another** without forgetting: **continual learning**.

## My part (Q2–Q3)

Q2 Can continual learning learn $\lambda$? Does it forget?

Q3 If the gain is small, why?

I ran and analysed the continual learning experiments: 6 methods $\times$ 3 orders $\times$ 3 seeds.

# Setup: four types in sequence

**Tasks** (learned in order): T1 text perception, T2 visual perception, T3 text relation, T4 visual relation (753 questions)

**Predictor**: small 2-layer network, 54 inputs from the two score curves

**Methods**: Naive (just keep training), EWC (protect important weights), LwF (imitate the old model), ER (replay old questions), kNN, Joint (train on everything; reference)

**Metrics**

- ACC: final recall@8 averaged over the 4 types
- FM (forgetting): how far old types drop from their best

**Baselines**

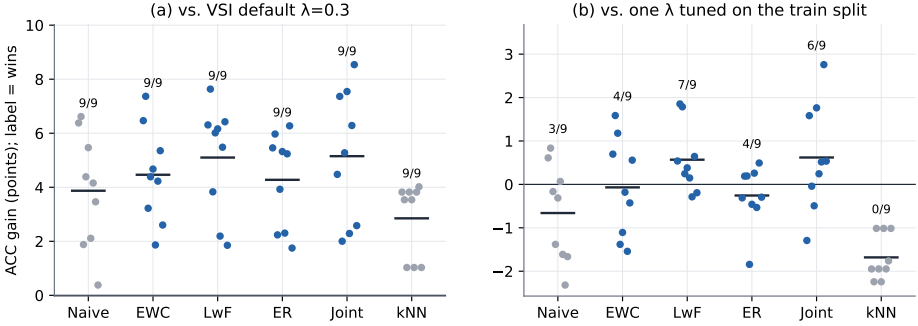
- default $\lambda = 0.3$
- **one $\lambda$ tuned on the training set**
- one $\lambda$ per type; oracle

# Q2: results over 9 runs

| Method | ACC ↑ | FM ↓ | vs. tuned |
|---|---|---|---|
| Default $\lambda = 0.3$ | 0.617 | – | −4.5 |
| One tuned $\lambda$ | 0.662 | – | – |
| One $\lambda$ per type | 0.677 | – | +1.4 |
| Naive | 0.656 | 0.029 | −0.7 |
| EWC | 0.662 | 0.023 | −0.1 |
| LwF | **0.668** | **0.015** | +0.6 |
| ER | 0.660 | 0.021 | −0.3 |
| Joint (ref.) | **0.668** | **0.015** | +0.6 |
| kNN | 0.645 | 0.005 | −1.7 |

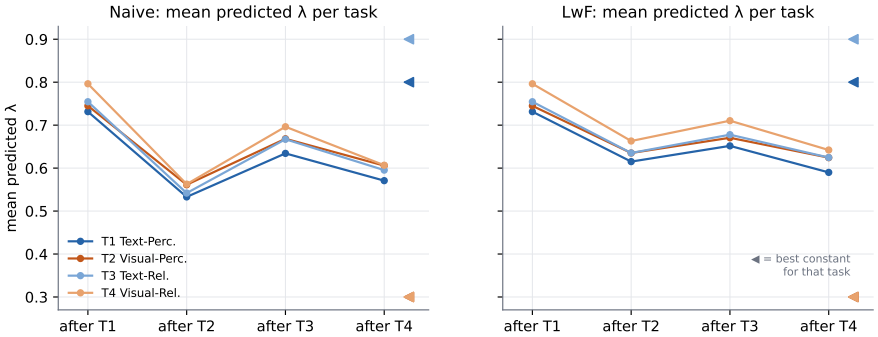
- Every method beats the default by **4–5 points**, in **9/9** runs
- Against **one tuned** $\lambda$: best gain only **+0.6**
- None reaches one $\lambda$ per type

# Q2: the right baseline matters




Each dot = one run. (a) vs. default: always better. (b) vs. one tuned $\lambda$: **about even**.

# Q3: one weight for every type




Average predicted $\lambda$ per type after each stage; triangles = best $\lambda$ for that type.
The lines overlap at $\approx$0.6: a **better global value**, not **different values per type**.

# Q3: why, and forgetting

Mohamed bin Zayed University of Artificial Intelligence

## Why it cannot learn the differences

1. **Sparse signal**: 62% of questions give the same result for every $\lambda$.
2. **No type information**: inputs only describe the score curves; adding the question text overfits.

## Why forgetting is small

All types get similar weights, so a new type overwrites little. LwF and Joint forget about **half** as much as Naive (0.015 vs. 0.029), but only in 6/9 runs.

# Takeaways from Q2–Q3

1. Continual learning **beats the default** weight in every run, by 4–5 points.
2. But it only **ties a tuned constant**, because it collapses to one weight for all question types.
3. Forgetting is **small** for every method; anti-forgetting methods help only weakly.
4. Next: give the predictor the question type, or start from one $\lambda$ per type and learn per-question corrections.

# Thank You

## Questions?

Longtao Jiang
with Jianxiang He and Makhmud Assentay

CV805 Course Project
Mohamed bin Zayed University of Artificial Intelligence



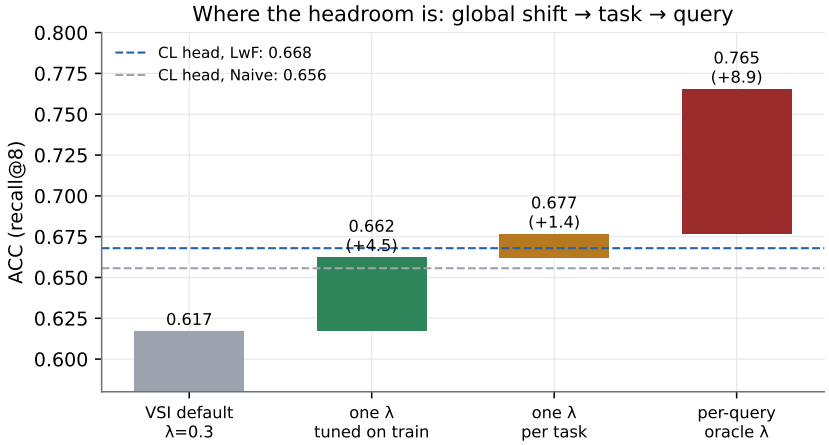
Mohamed bin Zayed
University of
Artificial Intelligence

- Outputs probabilities over 11 values $\lambda \in \{0, 0.1, \ldots, 1\}$; uses their **weighted average** $\bar{\lambda}$.
- "Best $\lambda$" labels carry little signal (most questions are flat), so we train it to rank the annotated seconds high:

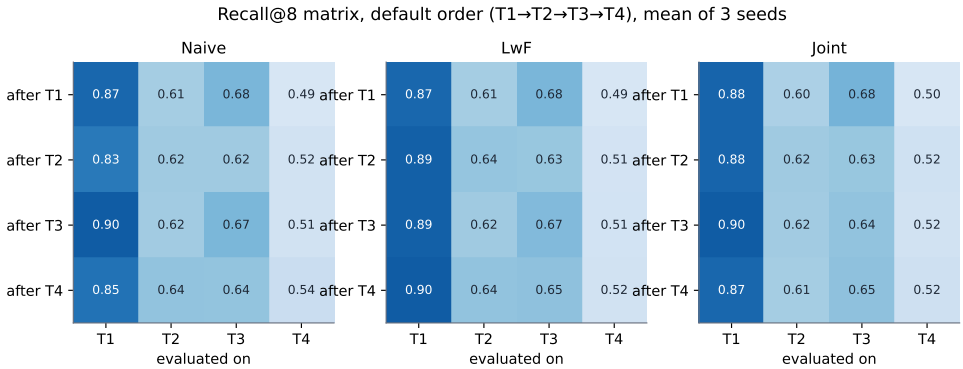
$$\mathcal{L} = -\log \sum_{t \in \mathrm{GT}} \mathrm{softmax}\left(S_{\bar{\lambda}}/\tau\right)_t, \quad \tau = 0.1$$

- 50 epochs per type, Adam, lr $10^{-3}$, batch 64. EWC weight 400; LwF temperature 2; ER buffer 200; kNN $k = 5$.
- Inputs standardized with the **first** type's statistics only (no leakage).

# Backup: where the gain is

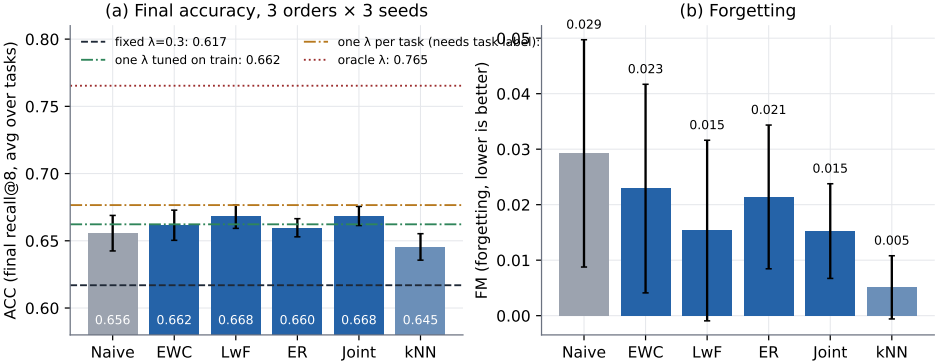


# Backup: recall matrices

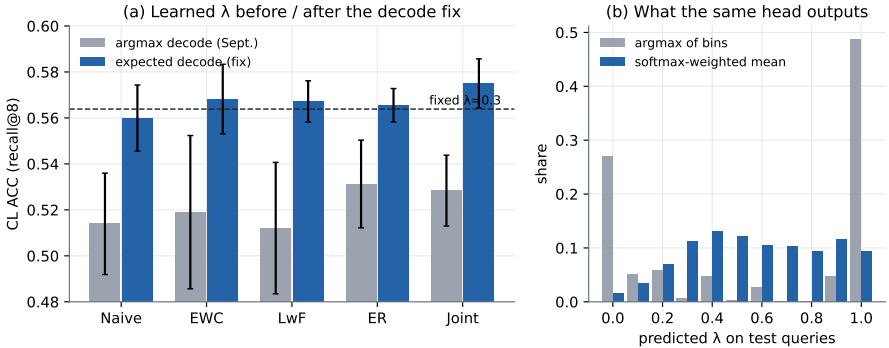



Row $i$: recall on each type after learning the $i$-th type (default order, 3 seeds).

# Backup: ACC and FM bars



# Backup: train/test mismatch




Trained on the average $\lambda$ but tested on the most likely value, which was 0 or 1. After the fix, learned weights went from 3–5 points below the default to above it.

# Backup: which inputs help

